PL/SQL Lab Exercises & Solutions

Exercise 1: Control Structures

Scenario 1

Question: Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

```
DECLARE
    CURSOR c_customers IS
        SELECT c.CustomerID, l.LoanID, l.InterestRate, c.DOB
        FROM Customers c
        JOIN Loans l ON c.CustomerID = l.CustomerID;

    v_age NUMBER;
BEGIN
    FOR r_cust IN c_customers LOOP
        v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, r_cust.DOB) / 12);

        IF v_age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE LoanID = r_cust.LoanID;
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Scenario 2

Question: Write a PL/SQL block that iterates through all customers and sets a flag IsVIP to TRUE for those with a balance over $10,000.

```
BEGIN
    FOR r_cust IN (SELECT CustomerID, Balance FROM Customers) LOOP
        IF r_cust.Balance > 10000 THEN
            DBMS_OUTPUT.PUT_LINE('Customer ID ' || r_cust.CustomerID || '
is promoted to VIP.');
            -- If column IsVIP exists:
            -- UPDATE Customers SET IsVIP = 'TRUE' WHERE CustomerID =
r_cust.CustomerID;
        END IF;
    END LOOP;
    COMMIT;
END;
/
```

Scenario 3

Question: Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.

```
DECLARE
    CURSOR c_due_loans IS
        SELECT c.Name, l.LoanID, l.EndDate
        FROM Loans l
        JOIN Customers c ON l.CustomerID = c.CustomerID
        WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30;
BEGIN
    FOR r_loan IN c_due_loans LOOP
        DBMS_OUTPUT.PUT_LINE('Reminder: Customer ' || r_loan.Name ||
                             ' has Loan ID ' || r_loan.LoanID ||
                             ' due on ' || TO_CHAR(r_loan.EndDate, 'YYYY-
MM-DD') || '.');
    END LOOP;
END;
/
```

Exercise 2: Error Handling

Scenario 1

Question: Write a stored procedure SafeTransferFunds that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (
    p_src_acc IN NUMBER,
    p_dest_acc IN NUMBER,
    p_amount IN NUMBER
) AS
    v_src_bal NUMBER;
    insufficient_funds EXCEPTION;
BEGIN
    SELECT Balance INTO v_src_bal FROM Accounts WHERE AccountID = p_src_acc
FOR UPDATE;

    IF v_src_bal < p_amount THEN
        RAISE insufficient_funds;
    END IF;

    UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID =
p_src_acc;
    UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID =
p_dest_acc;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer successful.');
EXCEPTION
    WHEN insufficient_funds THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds in source
account.');
    WHEN NO_DATA_FOUND THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: One or both account IDs do not
exist.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);
END;
/
```

Scenario 2

Question: Write a stored procedure UpdateSalary that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

```
CREATE OR REPLACE PROCEDURE UpdateSalary (
    p_emp_id IN NUMBER,
    p_percentage IN NUMBER
) AS
    v_dummy NUMBER;
    emp_not_found EXCEPTION;
BEGIN
    SELECT 1 INTO v_dummy FROM Employees WHERE EmployeeID = p_emp_id;

    UPDATE Employees
    SET Salary = Salary + (Salary * (p_percentage / 100))
    WHERE EmployeeID = p_emp_id;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee ID ' || p_emp_id || ' does
not exist.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Scenario 3

Question: Write a stored procedure AddNewCustomer that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
CREATE OR REPLACE PROCEDURE AddNewCustomer (
    p_cust_id IN NUMBER,
    p_name IN VARCHAR2,
    p_dob IN DATE,
    p_balance IN NUMBER
) AS
BEGIN
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_cust_id, p_name, p_dob, p_balance, SYSDATE);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('New customer added successfully.');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Customer with ID ' || p_cust_id || '
already exists.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Exercise 3: Stored Procedures

Scenario 1

Question: Write a stored procedure ProcessMonthlyInterest that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
BEGIN
    UPDATE Accounts
    SET Balance = Balance * 1.01,
        LastModified = SYSDATE
    WHERE AccountType = 'Savings';

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest processed for all savings
accounts.');
END;
/
```

Scenario 2

Question: Write a stored procedure UpdateEmployeeBonus that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department IN VARCHAR2,
    p_bonus_percent IN NUMBER
) AS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * (p_bonus_percent / 100))
    WHERE Department = p_department;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Bonus applied to department: ' || p_department);
END;
/
```

Scenario 3

Question: Write a stored procedure TransferFunds that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

```
CREATE OR REPLACE PROCEDURE TransferFunds (
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
) AS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID =
p_from_account FOR UPDATE;

    IF v_balance >= p_amount THEN
        UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID =
p_from_account;
        UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID =
p_to_account;
        COMMIT;
        DBMS_OUTPUT.PUT_LINE('Successfully transferred ' || p_amount);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Transfer failed: Insufficient Balance.');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Invalid account details.');
END;
/
```

Exercise 4: Functions

Scenario 1

Question: Write a function CalculateAge that takes a customer's date of birth as input and returns their age in years.

```
CREATE OR REPLACE FUNCTION CalculateAge (
    p_dob IN DATE
) RETURN NUMBER IS
    v_age NUMBER;
BEGIN
    v_age := FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);
    RETURN v_age;
END;
/
```

Scenario 2

Question: Write a function CalculateMonthlyInstallment that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (
    p_loan_amount IN NUMBER,
    p_interest_rate IN NUMBER,
    p_duration_years IN NUMBER
) RETURN NUMBER IS
    v_monthly_rate NUMBER;
    v_total_months NUMBER;
    v_installment NUMBER;
BEGIN
    v_monthly_rate := (p_interest_rate / 100) / 12;
    v_total_months := p_duration_years * 12;

    IF v_monthly_rate = 0 THEN
        v_installment := p_loan_amount / v_total_months;
    ELSE
        v_installment := p_loan_amount * (v_monthly_rate * POWER(1 +
v_monthly_rate, v_total_months)) /
                         (POWER(1 + v_monthly_rate, v_total_months) - 1);
    END IF;

    RETURN ROUND(v_installment, 2);
END;
/
```

Scenario 3

Question: Write a function HasSufficientBalance that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (
    p_account_id IN NUMBER,
    p_amount IN NUMBER
) RETURN BOOLEAN IS
    v_balance NUMBER;
BEGIN
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID =
p_account_id;
    IF v_balance >= p_amount THEN
        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/
```

Exercise 5: Triggers

Scenario 1

Question: Write a trigger UpdateCustomerLastModified that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

```
CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/
```

Scenario 2

Question: Write a trigger LogTransaction that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

```
CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (TransactionID, LogDate)
    VALUES (:NEW.TransactionID, SYSDATE);
END;
/
```

Scenario 3

Question: Write a trigger CheckTransactionRules that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

```
CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE
    v_balance NUMBER;
BEGIN
    IF :NEW.TransactionType = 'Withdrawal' THEN
        SELECT Balance INTO v_balance FROM Accounts WHERE AccountID
= :NEW.AccountID;
        IF :NEW.Amount > v_balance THEN
            RAISE_APPLICATION_ERROR(-20001, 'Withdrawal amount exceeds
available balance.');
        END IF;
    ELSIF :NEW.TransactionType = 'Deposit' THEN
        IF :NEW.Amount <= 0 THEN
            RAISE_APPLICATION_ERROR(-20002, 'Deposit amount must be
positive.');
        END IF;
    END IF;
END;
/
```

Exercise 6: Cursors

Scenario 1

Question: Write a PL/SQL block using an explicit cursor GenerateMonthlyStatements that retrieves all transactions for the current month and prints a statement for each customer.

```
DECLARE
    CURSOR GenerateMonthlyStatements IS
        SELECT c.Name, t.AccountID, t.TransactionID, t.TransactionDate,
t.Amount, t.TransactionType
        FROM Transactions t
        JOIN Accounts a ON t.AccountID = a.AccountID
        JOIN Customers c ON a.CustomerID = c.CustomerID
        WHERE TRUNC(t.TransactionDate, 'MM') = TRUNC(SYSDATE, 'MM');
BEGIN
    FOR r_stmt IN GenerateMonthlyStatements LOOP
        DBMS_OUTPUT.PUT_LINE('Customer: ' || r_stmt.Name ||
                             ' | Acc ID: ' || r_stmt.AccountID ||
                             ' | Trans ID: ' || r_stmt.TransactionID ||
                             ' | Date: ' || TO_CHAR(r_stmt.TransactionDate,
'YYYY-MM-DD') ||
                             ' | Type: ' || r_stmt.TransactionType ||
                             ' | Amount: $' || r_stmt.Amount);
    END LOOP;
END;
/
```

Scenario 2

Question: Write a PL/SQL block using an explicit cursor ApplyAnnualFee that deducts an annual maintenance fee from the balance of all accounts.

```
DECLARE
    CURSOR ApplyAnnualFee IS
        SELECT AccountID, Balance FROM Accounts FOR UPDATE;
    v_fee CONSTANT NUMBER := 50;
BEGIN
    FOR r_acc IN ApplyAnnualFee LOOP
        UPDATE Accounts
        SET Balance = Balance - v_fee,
            LastModified = SYSDATE
        WHERE CURRENT OF ApplyAnnualFee;
    END LOOP;
    COMMIT;
END;
/
```

Scenario 3

Question: Write a PL/SQL block using an explicit cursor UpdateLoanInterestRates that fetches all loans and updates their interest rates based on the new policy.

```
DECLARE
    CURSOR UpdateLoanInterestRates IS
        SELECT LoanID, InterestRate FROM Loans FOR UPDATE;
BEGIN
    FOR r_loan IN UpdateLoanInterestRates LOOP
        UPDATE Loans
        SET InterestRate = InterestRate + 0.5
        WHERE CURRENT OF UpdateLoanInterestRates;
    END LOOP;
    COMMIT;
END;
/
```

Exercise 7: Packages

Scenario 1: CustomerManagement

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal
NUMBER);
    PROCEDURE UpdateCustomerDetails(p_id NUMBER, p_name VARCHAR2);
    FUNCTION GetCustomerBalance(p_id NUMBER) RETURN NUMBER;
END CustomerManagement;
/

CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal
NUMBER) IS
    BEGIN
        INSERT INTO Customers(CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
    END AddCustomer;

    PROCEDURE UpdateCustomerDetails(p_id NUMBER, p_name VARCHAR2) IS
    BEGIN
        UPDATE Customers SET Name = p_name, LastModified = SYSDATE WHERE
CustomerID = p_id;
    END UpdateCustomerDetails;

    FUNCTION GetCustomerBalance(p_id NUMBER) RETURN NUMBER IS
        v_bal NUMBER;
    BEGIN
        SELECT Balance INTO v_bal FROM Customers WHERE CustomerID = p_id;
        RETURN v_bal;
    END GetCustomerBalance;
END CustomerManagement;
/
```